Proyecto: CargoBox (Sistema de Gestión Logística)

Equipo 14: Pedro Omar Cardona Alvarez, Kloudenn Jafed Jiménez Valdivieso, Leonardo Acosta Morales.

Herramienta utilizada: Gemini (Modelo LLM)

Bitacora obligatoria de IA

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
1	Etapa 1: Casos de Uso	Le pasamos las Historias de Usuario y el diagrama BPMN de CargoBox y le pedimos que nos sacara 5 casos de uso principales y de excepción en una tabla.	Nos hizo bien la lista inicial cubriendo los pasos lógicos, desde que llega el paquete a sucursal hasta que se entrega.	Supuso el formato de la tabla y se le olvidó poner las columnas de "Información de entrada" y "Objetivo" que pedía explícitamente la actividad.	No aceptamos esa primera tabla. Le pedimos que la volviera a hacer agregando los datos que faltaban para cumplir con todo. También nosotros validamos manualmente que el caso de "ausencia" respetara nuestra regla de los 2 intentos.
2	Etapa 2: Patrón de Lógica	Le dijimos que comparara Transaction Script, Table Module y Domain Model usando las reglas de CargoBox (como la de no cancelar paquetes que ya salieron).	Explicó muy bien las ventajas de protección de datos del Domain Model.	Asumió y argumentó que usar Domain Model para absolutamente todo el sistema era lo mejor por ser la arquitectura más "moderna y segura".	Nos dimos cuenta de que usar Domain Model para todo iba a hacer muy lentos los reportes. Decidimos hacer una mezcla por nuestra cuenta: Domain Model para el control diario de los paquetes y Table Module solo para sacar los reportes rápidos de la gerencia.
3	Etapa 3: Mapa de Dominio	Le pasamos nuestro glosario limpio y le pedimos que identificara cuáles eran los Aggregates, Entidades y Value Objects, y que justificara por qué.	Atinó perfecto en que el "Envío" era el Aggregate Root principal del sistema.	Se equivocó en el análisis de prioridades al sugerir que el "Seguro" era una Entidad independiente.	Revisamos el BRD y vimos que a CargoBox solo le importa el "monto del seguro", no rastrearlo por separado. Así que corregimos a la IA y nosotros cambiamos el Seguro a un Value Object para no hacer el diagrama tan complejo.
4	Etapa 4: Service Layer	Con los 5 casos de uso que ya teníamos, le pedimos que diseñara los servicios (Service Layer) para coordinar todo el trabajo.	Nos dio muy buenos nombres para los servicios (Application Services).	Metió las reglas de negocio ahí mismo en el servicio (por ejemplo, puso la lógica para contar los intentos de entrega directo en el Application Service).	Esto rompía la regla de separar responsabilidades. Corregimos el diseño para obligar a que el servicio solo coordine y que la regla matemática de contar los intentos se moviera directamente adentro del objeto "Envío".

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
5	Etapa 5: Transacciones	Le pedimos que diseñara los repositorios y que nos dijera exactamente dónde empezaba y terminaba una transacción (Unit of Work) en nuestro sistema.	Comprendió bien la idea de agrupar los cambios para que no quedaran datos a medias.	Nos dio una propuesta muy técnica con código SQL revuelto con las reglas de negocio y un manejo de errores súper abstracto.	Quitamos el SQL porque el dominio no debe saber de bases de datos. Nosotros aterrizamos la transacción a la vida real: inicia cuando el repartidor le pica a "Ausente" en su celular; si el internet falla, se hace un Rollback.
6	Etapa 6: CQRS	Le pedimos aplicar CQRS para separar la parte de escribir/actualizar estados de la parte de consultar el rastreo de los paquetes.	Explicó cómo la separación ayudaría a los clientes a consultar sus paquetes más rápido.	Nos aconsejó usar dos bases de datos físicas distintas (una para leer y otra para guardar) para que el sistema fuera súper rápido.	Nos pareció una exageración y muy costoso para el tamaño de esta empresa. Decidimos usar un CQRS básico: usamos una sola base de datos, pero en el código sí separamos las vistas de lectura de los modelos de escritura.
7	Etapa 7: Principios SOLID	Le dimos todo lo que llevábamos diseñado y le pedimos que buscara errores según los principios SOLID y nos propusiera soluciones.	Encontró problemas de acoplamiento reales en el código base que no habíamos notado.	El problema fue que nos dio ejemplos de tiendas en línea para solucionarlo, en lugar de usar el lenguaje de paquetería del proyecto.	Tomamos la idea pero la adaptamos a CargoBox. Para el principio Abierto/Cerrado, nosotros diseñamos la interfaz AsignadorCentroDistribucion para quitar código rígido y poder asignar rutas por código postal fácilmente.
8	Etapa 8: Diagramas y ADR	Le pedimos que nos generara el código base en PlantUML para el diagrama de clases y los de secuencia basándose en las etapas anteriores.	Generó la sintaxis correcta de PlantUML, lo cual nos ahorró mucho tiempo de escritura.	El código que generó estaba todo amontonado; visualmente puso la base de datos (infraestructura) revuelta con la aplicación.	Editamos el código UML a mano para separar visualmente bien las 3 capas (Aplicación, Dominio, Infraestructura). También revisamos los ADRs para que no sonaran teóricos, sino con las justificaciones reales que platicamos.
9	Etapa 9: Validación Final	Le pedimos que respondiera las preguntas finales de validación de la guía, checando que todo cuadrara con nuestro BRD de la Unidad 2.	Las respuestas estaban técnicamente correctas y trazaban bien todo con nuestros documentos.	Usó un lenguaje demasiado robótico, técnico y con párrafos larguísimos que no sonaban a nosotros.	Reescribimos todas las justificaciones con nuestras propias palabras para que sonaran naturales y fáciles de explicar, asegurándonos de poder defender en clase cómo nuestro diseño protege las reglas de CargoBox.

Lo que aprendimos al usar la IA como equipo:

- **Facilidad para la redacción de trabajos largos:** Nos dimos cuenta de que la IA es una herramienta increíble para quitarnos de encima el trabajo más pesado. En entregables tan largos como este, la IA nos ahorró muchísimas horas al momento de estructurar tablas inmensas, armar el esqueleto de los documentos y generar la base de textos extensos.
- **Su gran precisión:** Nos sorprendió lo exacta que puede ser la herramienta cuando le pasas la información correcta. Al momento de pedirle que mapeara nuestros casos de uso o que buscara dónde encajaban los principios SOLID en nuestro código, la IA tuvo una precisión excelente para detectar los detalles que le pedimos sin perder el hilo.
- **Redacta como un libro (Es necesaria una intervención humana):** Nuestra mayor queja es que la IA escribe de una forma muy seca como si fuera un libro. Para obtener el resultado deseado es necesario que tenga intervención humana para poder corregir la forma en la cual se redacta la información y así se pueda utilizar de mejor manera en el documento que estamos trabajando